

Lecture 16

Sorting

- Sorting is an algorithm that puts elements of array or list into an order. The most frequently used orders are numerical order and lexicographical order, and either ascending or descending.
- Efficient sorting is important for optimizing the efficiency of other algorithms such as **search** and **merge** algorithms that require input data to be in sorted order.
- The output of any sorting algorithm must satisfy two conditions:
 1. The is in monotonic order
 2. The output is a permutation (a reordering, yet retaining all of the original elements) of the input.

Why sorting algorithms are important?

- Organise large datasets for easier handling and printing
- Enabling quick access to the k -th smallest or largest elements
- Making binary search possible for fast lookups in sorted data

Important points

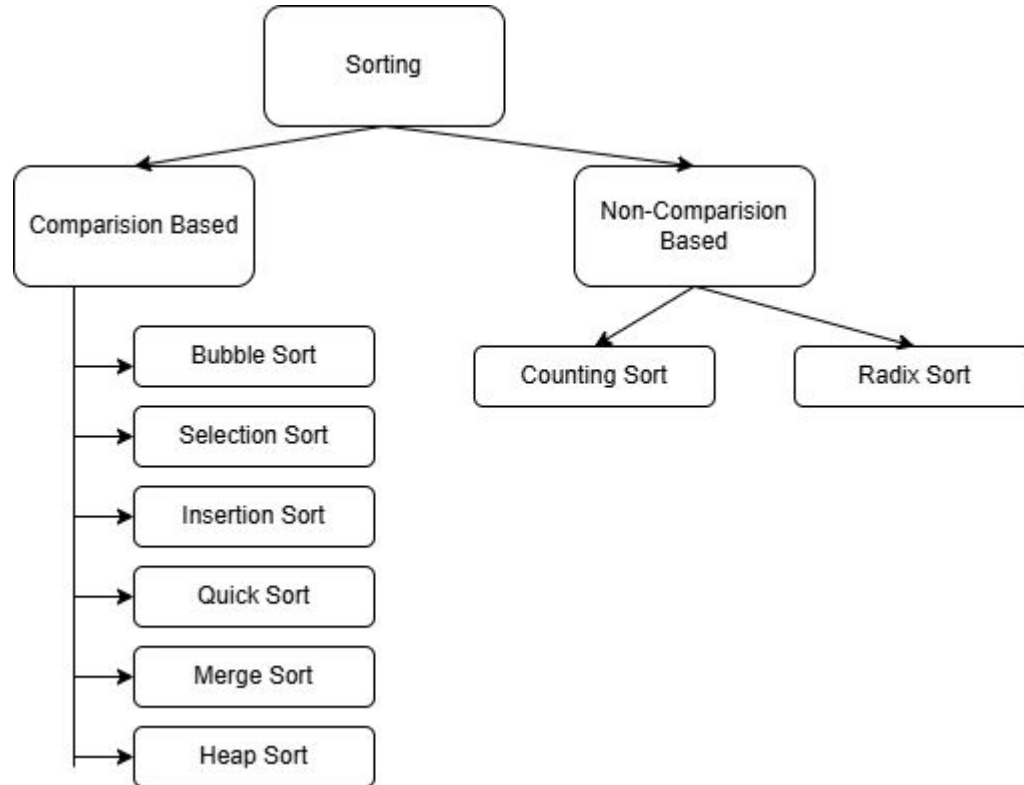
In-place Sorting: It uses **constant space** for producing the output.

Example: selection sort, bubble sort, insertion sort, heap sort etc.

Stable Sorting: when two same elements appear in the **same order** in sorted data as in the original array.

Example: Merge Sort, Insertion Sort , Bubble Sort etc.

Types of Sorting



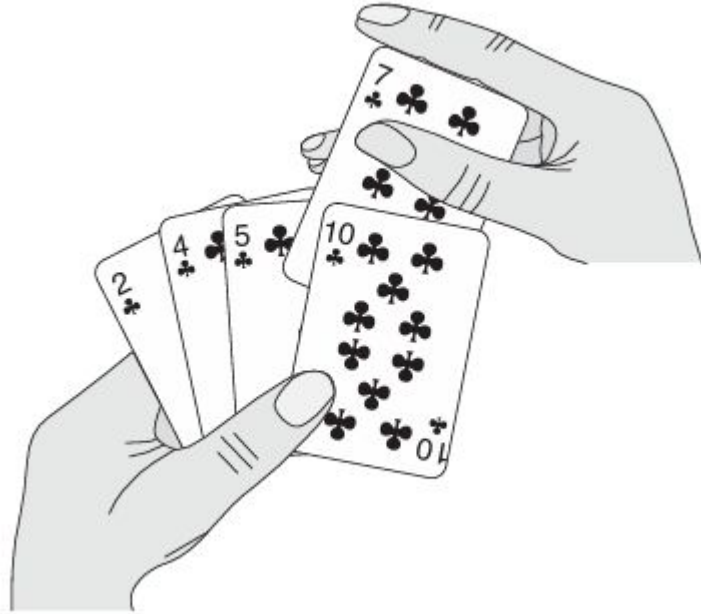
Bubble Sort

- Bubble sort repeatedly swapping the adjacent element if they are in the wrong order.
- The worst case scenario for bubble sort is when array is sorted in reverse order.
- This requires $n-1$ rounds/passes and in each round adjacent elements are swapped, making it highly inefficient for large dataset.
- No of comparisons = $n(n-1)/2$
- No of swaps = $n(n-1)/2$

Insertion Sort

- Insertion Sort is an efficient algorithm for sorting a small number of elements. Insertion sort works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table. We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards already in the hand, from right to left, as illustrated in Figure. At all times, the cards held in the left hand are sorted, and these cards were originally the top cards of the pile on the table.

continue...



continue...

- Insertion sort shifting takes place rather than swap.
- Insertion sort give $O(n)$ or linear time complexity in best case(when array is already sorted).

{1,| 2, 3, 4, 5}

For placing 2 at its right place only one comparison is needed

For placing 3 at its right place only one comparison is needed

For placing 4 at its right place only one comparison is needed

For placing 5 at its right place only one comparison is needed

So total $n-1$ comparisons are needed if input is already sorted. Therefore, time complexity for sorted array in insertion sort is $O(n)$.

continue...

- Insertion Sort has worst case time complexity when input is sorted in reverse order.

{4, 3, 2, 1}

For placing 3 at its right place only 1 comparison is needed

For placing 2 at its right place 2 comparisons are needed

For placing 1 at its right place 3 comparisons are needed

Total no of comparison = $1 + 2 + 3 + \dots + (n-1) = n(n-1)/2 = O(n^2)$

Advantages of Insertion Sort

- Insertion sort is a stable sorting algorithm
- Insertion sort is a inplace sorting algorithm
- Efficient for small list or nearly sorted list

Disadvantage of Insertion Sort

- Insufficient for large lists